Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- Dersin kaynakları ve konu akışı
  - Slayttaki kaynaklar (ekrandan, 00:25:50): Khalid Sayood, *Introduction to Data Compression*; J. Gibson, T. Berger, T. Lookabaugh, *Digital Compression for Multimedia*; D. Salomon, *Data Compression*. Derslerde ağırlıklı olarak Salomon'un kitabı kullanılır; bazı konular anlatımı daha kolay olan başka kaynaklardan alınır.
  - Konular (ekrandan, 00:40:00): veri sıkıştırma nedir; temel teknikler (sezgisel yöntemler, Run Length Coding, Move-To-Front); hata bulma ve düzeltme (eşlik bitleri, CRC); statik kodlar (prefix kodlar, Golomb kodu); istatistiksel yöntemler (Shannon-Fano, Huffman ve türevleri, MNP ve türevleri, aritmetik kodlama, PPM); sözlüksel yöntemler (LZ ve türevleri). Hoca, hata bulma ve düzeltmeyi temel tekniklerin önüne alabileceğini söyledi.
  - Huffman'ın türevleri: statik, adaptif ve yarı adaptif Huffman ile Huffman başlık (header) bilgisini kısaltan kodlamalar. MNP (Microcom Network Protocol) ailesinin 5 ve 7 gibi sürümleri vardır.
  - İstatistiksel yöntemler, sembollerin (karakter, piksel) olasılık bilgisini kullanır. Sözlüksel yöntemler olasılık yerine verideki tekrar eden desenleri (pattern) bulup kodlar; LZ adı Lempel ve Ziv'in soyadlarından gelir.
  - Ayrıca bir bilgi teorisi (information theory) bölümü olacak; bu bölümün sıkıştırma için olmazsa olmaz kavramı entropidir.
- Veri sıkıştırma nedir?
  - Girdi akışını (input stream) alıp boyutunu küçülterek çıktı akışına (output stream) yazmaktır (ekrandan, 00:50:00). Girdi akışına kaynak akış (source stream), ham (raw) ya da orijinal veri de denir; bir dosya ya da bir tampon (buffer) alanı olabilir.
  - Sıkıştırılan veri açılarak (decompress) girdi akışı yeniden elde edilir; birebir aynı ya da biraz eksik olabilir.
  - Sıkıştırmanın iki temel amacı vardır: fiziksel alandan kazanç (100 birimlik veriyi 30 birimlik yerde saklamak) ve iletimde hız (A noktasından B noktasına daha kısa sürede göndermek ya da indirmek).
- Sabit uzunluklu kod (fixed size code)
  - Her sembolü aynı sayıda bitle kodlar. Örnek: ASCII, her karakter 8 bit. Kodlaması ve çözmesi kolaydır ama ideal değildir: Türkçede en sık "a", en seyrek "j" geçtiği hâlde ikisi de 8 bittir.
- Değişken uzunluklu kod (variable size code)
  - Sıkıştırmanın temel ilkesi: sık geçen sembol az bitle, seyrek geçen sembol çok bitle kodlanır. Kod uzunluğu sembolün kullanım sıklığıyla ters orantılıdır.
  - Örnek (ekrandan, 00:53:40): bir metinde 30 "a", 2 "j" var. Sabit uzunlukta $30 \cdot 8 + 2 \cdot 8 = 240 + 16 = 256$ bit. "a" 3 bit, "j" 9 bit ile kodlanırsa $30 \cdot 3 + 2 \cdot 9 = 90 + 18 = 108$ bit.
  - Bit uzunlukları elle seçilmez; Huffman, Shannon-Fano, aritmetik kodlama, LZW, PPM gibi algoritmaların ürettiği kodlar kullanılır.
  - Sembol bir karakter, bir karakter çifti ya da görüntüde bir piksel olabilir; kodlanacak girdi birimidir.
- Artıklık (redundancy)
  - Veride hâlâ sıkıştırılabilecek kısım kaldığını gösterir. Bir sembolü 3 bite indirebilen bir algoritma varken 8 biti 5 bite indiren algoritma kullanılırsa artıklık büyük kalır

(ekrandan, 00:59:10: $8 \to 5 \to 3$).
  - Artıklık sıfıra yaklaşıyorsa seçilen algoritma o veri için iyidir. Büyükse sıkıştırılmış veriye bir algoritma daha uygulanabilir; art arda sıkıştırma artıklık sıfıra yaklaşana kadar anlamlıdır.
  - Artıklığın formülü entropi anlatıldıktan sonra verilecek.
- Temel kavramlar (ekrandan, 01:30:00)
  - Compressor / encoder / kodlayıcı: girdi akışını sıkıştırıp çıktı akışına yazan algoritma. Decompressor / decoder / kod çözücü: bunun ters işlemini yapan algoritma. Codec: ikisinin birleşimi.
  - Unencoded / raw / original data: işlenmemiş veri. Encoded / compressed: sıkıştırılmış veri.
  - Non-adaptive (adaptif olmayan) sıkıştırma: yalnız belli tipte veriyi sıkıştırır; parametreleri değişmez. Örnek: faks (facsimile) sıkıştırması; faks metni görüntü olarak gönderir ve siyah-beyaz piksellerle çalışır, ses verisine uygulanamaz.
  - Adaptive (adaptif) sıkıştırma: her tür veride çalışır; parametreler ve kodlar işlem sırasında değişebilir. Örneğin "a"nın kodu metnin başında, ortasında ve sonunda farklı olabilir.
  - Statik, yarı dinamik (semi-adaptive), dinamik: statikte kodlar baştan bellidir ve sonuna kadar aynı kalır (ASCII tablosu gibi). Yarı dinamik iki aşamalıdır: önce sembol frekansları çıkarılır, sonra kod atanır. Dinamikte kodlar akış boyunca değişir, iletişim anında hızın önemli olduğu yerlerde kullanılır; alıcı her değişikliği anında yakalayarak açar.
  - Lossy (kayıplı): $X \to Y \to X'$, $X \neq X'$. Lossless (kayıpsız): $X \to Y \to X$, birebir aynı (ekrandan, 01:40:00).
  - Simetrik sıkıştırma: kompresör ve dekompresör aynı algoritmayı ters yönde kullanır. Asimetrik sıkıştırma: iki taraf farklı algoritmadır; biri karmaşık ve yavaş, diğeri basit ve hızlıdır.
  - Stream mode / block mode: stream modda veri bit, bayt ya da kelime kelime okunup hep aynı işlem uygulanır. Blok modda veri 512 ya da 1024 baytlık bloklar hâlinde okunur, her bloğa ayrı sıkıştırma uygulanabilir ve blok başına çözme bilgisi yazılır. Burrows-Wheeler dönüşümü blok modda çalışır.
  - Fiziksel ve mantıksal sıkıştırma: fiziksel sıkıştırma veriyi yalnız 1 ve 0 olarak görür, sıkıştırılmış veri açılmadan anlaşılmaz. Mantıksal sıkıştırma verideki desenleri bir kodla değiştirir (örneğin bir programdaki bütün `printf` yerine bir kod), sıkıştırılmış dosyaya bakınca desen tanınabilir; konuya özel (ad hoc) uygulamalardır.
- Sıkıştırma performansı
  - Sıkıştırma oranı (ekrandan, 02:03:30): oran $= \dfrac{\text{çıktı akışının boyutu}}{\text{girdi akışının boyutu}}$. 100 birimlik dosya 70'e inerse oran $0{,}7$, performans $(1 - 0{,}7) \cdot 100 = \%30$. 30'a inerse oran $0{,}3$, performans $\%70$.
  - Oran 1'den küçük olmalıdır; 1'den büyükse (örneğin $1{,}5$) veri sıkıştırılmamış, büyütülmüştür.
  - Metinde birim bpc (bits per character): bir karakter ortalama kaç bitle kodlanıyor. Bu aynı zamanda entropinin birimidir. Görüntüde birim bpp (bits per pixel).
- Derlem (corpus)
  - Algoritmaların karşılaştırılması için herkesin kullandığı standart veri topluluğu. Metin için Calgary ve Canterbury derlemleri (ekrandan, 02:06:40); Canterbury daha geniştir ve daha yeni dosya türlerini içerir. Görüntü için hocanın "UC" diye andığı bir veri seti vardır.
  - Doküman sınıflandırmada Reuters ve 20 Newsgroups derlemleri örnek verildi.
- Görüntü sıkıştırmada performans (ekrandan, 02:20:00 ve 02:22:55)
  - $M \times N$ boyutlu I görüntüsü sıkıştırılıp açılınca $\hat{I}$ elde edilir; boyut değişmez, kullanılan renk (sembol) sayısı azalır (örneğin 256 renkten 64 renge).
  - error$(r, c) = \hat{I}(r, c) - I(r, c)$, e_{RMS}, SNR_{RMS} ve $PSNR$ ölçütleri kullanılır (formüller Detaylı Açıklamalar bölümünde).
  - e_{RMS} küçükse açılan görüntü orijinale yakındır. SNR ile e_{RMS} ters orantılıdır.

Hocanın Özellikle Vurguladığı Kısımlar

- Sıkıştırılan veri açılabilmelidir
  - Açılamayan veri sıkıştırılmış değil kaybedilmiştir. Hoca bunu kilitlenip içine girilemeyen bir eve benzetti; her kompresörün bir dekompresörü olmalıdır.
- Fixed size code, redundancy, variable size code
  - Hoca bu üç kavramı "çok önemli" diye niteledi; dersin geri kalanında sürekli kullanılacaklar.
- Entropi
  - Bilgi teorisi bölümünde sıkıştırmanın olmazsa olmaz kavramı olarak işlenecek. Artıklığın formülü ve bpc birimi entropiye dayanır.
- Art arda sıkıştırmanın sınırı
  - Arka arkaya algoritma uygulayarak veriyi bir bayta indirmek mümkün değildir; durma noktası artıklığın sıfıra yaklaşmasıdır.
- Tek bir "en iyi" algoritma yoktur
  - Bazı algoritmalar metinde, bazıları görüntüde, bazıları çok tekrar eden desenlerde daha iyidir. Seçim veri tipine, hıza (sıkıştırma ya da açma) ve sıkıştırma başarısına göre yapılır; kendi alanındakilerin ne kullandığına bakmak da bir yoldur.
- Kayıplı ve kayıpsız sıkıştırmanın yeri
  - Metin kayıpsız sıkıştırılır: "10.000 dolar" içindeki sıfırlar kaybolup "100 dolar" olursa sonuç felakettir. Ses ve görüntü kayıplı sıkıştırılabilir, çünkü kulağın duymadığı frekanslar ve gözün ayırt edemediği renkler atılabilir.
  - Metinde de istisnalar vardır (fazla boşlukları tekleme, format bilgisini atma, programdaki boş satırları silme); bunlar kişiye ve konuya özel (ad hoc) kayıplı uygulamalardır.
- Sıkıştırma oranı ile sıkıştırma performansı karıştırılmamalı
  - Oran $0{,}7$ ise performans $\%30$'dur.
- Performans ölçümü standart derlem üzerinde yapılmalı
  - Kendi hazırladığı veriyle "%90 başarı" diyen bir yönteme kimse inanmaz.

Kısa Tekrar Notları

- Sıkıştırma: girdi akışı → kompresör → çıktı akışı; dekompresör ters işlemi yapar.
- Amaç: alan ve hız kazancı.
- Sabit uzunluklu kod: her sembol aynı bit (ASCII 8 bit). Değişken uzunluklu kod: sık sembol az bit.
- 30 "a" + 2 "j": sabit 256 bit, değişken (3 ve 9 bit) 108 bit.
- Artıklık sıfıra yakınsa algoritma iyi seçilmiştir; büyükse bir algoritma daha uygulanabilir.
- Codec = encoder + decoder.
- Non-adaptive: belli veri tipi (faks). Adaptive: her veri, kodlar değişebilir. Semi-adaptive: önce frekans, sonra kod.
- Statik / yarı dinamik / dinamik kodlama.
- Lossy: $X \neq X'$; lossless: $X = X'$. Metin kayıpsız, ses ve görüntü kayıplı.
- Simetrik: aynı algoritma ters yönde. Asimetrik: farklı algoritmalar (arşiv: yavaş sıkıştırma, hızlı açma; günlük yedekleme: hızlı sıkıştırma, yavaş açma).
- Stream mode: bit/bayt bazında; block mode: 512/1024 baytlık bloklar (Burrows-Wheeler).
- Fiziksel sıkıştırma içeriğe bakmaz; mantıksal sıkıştırma desen yerine kod koyar.
- Sıkıştırma oranı = çıktı / girdi; < 1 olmalı. Performans $= (1 - \text{oran}) \cdot 100$.
- Metin: bpc; görüntü: bpp.
- Derlem: Calgary, Canterbury (metin); Reuters, 20 Newsgroups (sınıflandırma).
- Görüntü: e_{RMS} küçük, SNR büyük olmalı.

Detaylı Açıklamalar

- **Sabit ve değişken uzunluklu kod.** ASCII'de "a" da "j" de, nokta da ünlem de 8 bittir; kodların sabit olması kodlamayı ve çözmeyi kolaylaştırır ama sembollerin sıklık farkını hiç kullanmaz. Hocanın örneğinde (ekrandan, 00:53:40) metinde 30 "a" ve 2 "j" vardır. ASCII ile $30 \cdot 8 = 240$ ve $2 \cdot 8 = 16$, toplam 256 bit harcanır. Sık geçen "a"ya 3 bit, seyrek geçen "j"ye 9 bit verilirse $30 \cdot 3 = 90$ ve $2 \cdot 9 = 18$, toplam 108 bit harcanır. "j" daha uzun kodlansa da toplam neredeyse yarıdan aza iner. Örnekteki 3 ve 9 bit hocanın keyfî

seçtiği değerlerdir; gerçekte bu uzunlukları Huffman, Shannon-Fano, aritmetik kodlama gibi algoritmalar belirler.

- **Artıklık.** Hoca artıklığı şöyle anlattı: "a"yı 8 bitten 3 bite indirebilen bir algoritma varken 5 bite indiren bir algoritma seçildiyse veride hâlâ atılabilecek fazlalık vardır, artıklık büyüktür (ekrandan, 00:59:10). Bir metindeki ardışık boşluklardan yalnız birkaçını silen algoritma artıklığı büyük bırakır; kelimeler arasında tek boşluk kalana kadar silen algoritma artıklığı küçültür. Artıklık büyükse sıkıştırılmış verinin üzerinden başka bir algoritma daha geçirilebilir. Artıklık sıfıra yaklaştığında yeni bir yöntem uygulamak anlamsızdır. Biçimsel tanım entropiden sonra verilecek.
- **Statik, yarı dinamik, dinamik.** Statik kodlamada kodlar sıkıştırma başlamadan bellidir ve sonuna kadar değişmez; hazır ASCII tablosundan kod almak buna örnektir. Yarı dinamik kodlamada veri önce okunur, sembollerin frekansı çıkarılır, sonra sık sembollere kısa kod atanır. Dinamik kodlamada aynı sembolün kodu akış boyunca değişir; hoca İstanbul'dan Ankara'ya hızlı veri gönderme örneğinde "a"nın önce bir kodla, ilerleyen zamanlarda farklı kodlarla gönderilebileceğini ve alıcının her değişikliği anında yakalayarak açtığını anlattı (ekrandan, 01:40:00). Bu üç türün çalışması Huffman kodlarında ayrıntılı gösterilecek.
- **Asimetrik sıkıştırmanın iki yönü.** Sıkıştırmanın yavaş, açmanın hızlı olması gereken durum arşiv uygulamalarıdır: bir rapor arşive bir kez konur ama yüzlerce kişi gün içinde indirir. Tersi, sıkıştırmanın hızlı, açmanın yavaş olabileceği durumdur: bankalarda her gün sonunda alınan yedek (backup). Yedekleme her gün yapılır, geri yükleme ise yılda bir iki kez gerekir. Hoca buradaki "uzun" ifadesinin saatler değil, 5 dakikalık işin tersinin 15 dakika sürmesi gibi düşünülmesi gerektiğini söyledi.
- **Sıkıştırma oranı ve performans.** 100 birimlik bir dosya 70 birime inerse oran $70/100 = 0{,}7$ olur; atılan kısım 30 birimdir, yani performans $(1 - 0{,}7) \cdot 100 = \%30$'dur. Dosya 30 birime inseydi oran $0{,}3$, performans $\%70$ olurdu (ekrandan, 02:03:30). Oranın 1'den büyük çıkması, yanlış algoritma seçilip dosyanın büyütüldüğü anlamına gelir.
- **Görüntüde performans.** Görüntüde çıktı/girdi oranı tek başına yeterli değildir, çünkü kayıplı sıkıştırma görüntüyü bozabilir; açılan görüntünün orijinale ne kadar yakın olduğu da ölçülmelidir. Ekrandaki tanımlar, hata farklarının karesi alınmış biçimde (ekrandan, 02:20:00 ve 02:22:55; düzeltme aşağıdaki notta):

$$\text{error}(r,c) = \hat{I}(r,c) - I(r,c)$$

$$e_{RMS} = \sqrt{\frac{1}{NM}\sum_{r=0}^{N-1}\sum_{c=0}^{M-1}\left[\hat{I}(r,c) - I(r,c)\right]^2}$$

$$SNR_{RMS} = \sqrt{\frac{\sum_r \sum_c \hat{I}(r,c)^2}{\sum_r \sum_c \left[\hat{I}(r,c) - I(r,c)\right]^2}}$$

$$PSNR = 10\log_{10}\frac{(L-1)^2}{\frac{1}{NM}\sum_r \sum_c \left[\hat{I}(r,c) - I(r,c)\right]^2}$$

  Burada L, görüntüdeki renk (gri seviye) sayısıdır.

  - Ders sonrası not: Ekranda "total error" farkların kare alınmadan toplamı olarak yazıldı ve e_{RMS} ile SNR bu toplamla kuruldu. "Root mean square" adı gereği hata farklarının karesi alınır; yukarıdaki formüller büyük olasılıkla doğru olan kareli biçimde yazılmıştır. Hoca da formülleri daha düzgün yazıp paylaşacağını söyledi.
- **Derlemler.** Hoca Calgary ve Canterbury derlemlerinin ikisinin de Kanada'daki üniversitelerde hazırlandığını söyledi.

  - Ders sonrası not: Calgary derlemi büyük olasılıkla Kanada'daki University of Calgary'de, Canterbury derlemi ise büyük olasılıkla Yeni Zelanda'daki University of Canterbury'de hazırlanmıştır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.